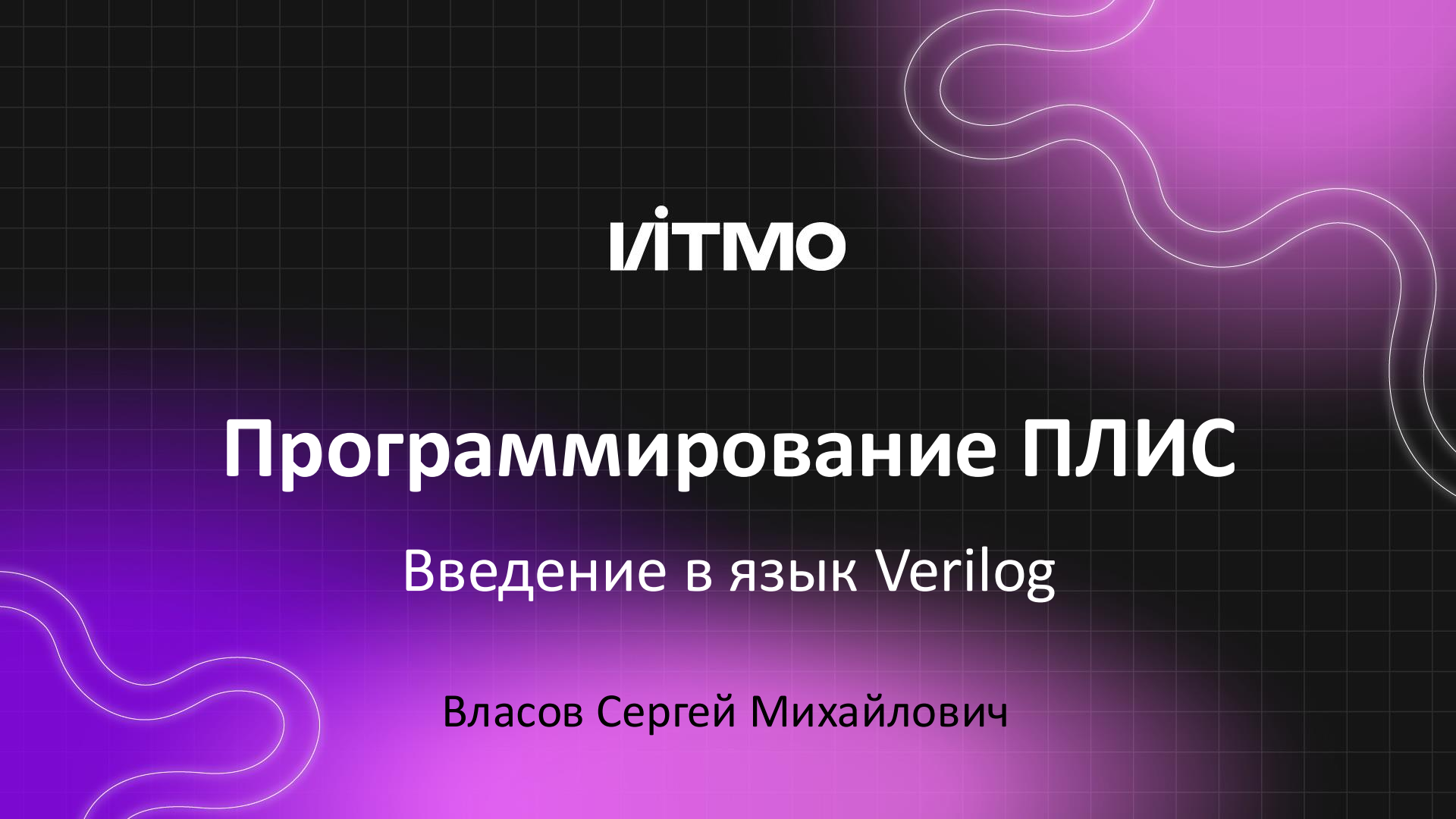




ІТМО

Программирование ПЛИС

Введение в язык Verilog

Власов Сергей Михайлович

Для успешного запуска своего кода он должен обладать определенной структурой:



```
// первым должно быть ключевое слово module после которого идет его
имя
module comb
(
// внутри скобок указываются все входы и выходы, которые будут
подключены к модулю
input wire [7:0] a,b,
output reg c
);

// тело модуля, здесь прописывается вся его логика

// в конце основной логики должно быть ключевое слово endmodule
endmodule
```

Классы типов данных



- **типы для моделирования аппаратуры,**
- стандартные арифметические типы данных.

Значения сигналов:

- 0 – логический ноль, или ложь;
- 1 – логическая единица, или истина;
- x – неопределенное значение. К примеру, значение регистра в начальный момент симуляции (до сброса или первой записи в регистр);
- z – состояние с высоким импедансом. Чаще всего сигнал принимает это значение, если он никуда не подключен – “обрыв провода”;

Основные типы данных:

- wire,
- reg,
- logic (только в SystemVerilog, но по сути тот же reg).

Данные типы могут использоваться как шины и N-разрядные регистры если их записывать следующим образом:

```
wire [3:0] a;
```

В такой записи `a[3:0]` представляет собой 4-битную шину. Биты, от старшего к младшему, записываются так: `a[3]`, `a[2]`, `a[1]` и `a[0]`.

Такой порядок битов называется little-endian, т.к. младший бит имеет наименьший битовый номер.

Мы могли бы назвать шину `a[4:1]`, и тогда `a[4]` был бы старшим.

Или мы могли бы написать `a[0:3]`, и тогда порядок битов от старшего к младшему был бы следующим: `a[0]`, `a[1]`, `a[2]` и `a[3]`. Такой порядок битов называется big-endian.



Процедурные блоки

Процедурный блок - процедура, срабатывающая по определенному событию.

Можно использовать C подобные управляющие конструкции:

- if... else..
- for
- do... while..
- case (аналог switch)



Строка **always** @(posedge clk) называется списком чувствительности.



Какие могут быть события:

- Изменение состояния сигнала - always @(a or b)
- posedge – возрастающий фронт сигнала – always @(posedge clk)
- negedge – падающий фронт сигнала - always @(negedge clk)
- always @* - выполнение, когда изменяются значения сигналов в правой части присваивания внутри блока always

При описании комбинационной схемы с помощью процедурного блока **always** необходимо помнить, что значения всех изменяемых в блоке сигналов, должны полностью определяться сигналами в списке чувствительности. В противном случае синтезатору придется вставить в схему элементы памяти — защелки (latches).

Процедурные блоки

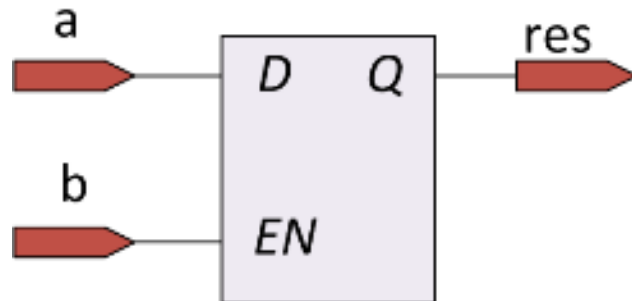
```
always @*  
begin  
    if(a & b) res = 1;  
    else if (!a & b) res = 0;  
end
```



В процедурном блоке не описано, что будет в случае если $b = 0$. Если $b = 0$ значение res не должно меняться, не зависимо от значения a . Поэтому будет просинтезирована следующая схема:

Как правило, появление защелок в схеме означает ошибку в коде.

Их присутствие может сильно ухудшить работу схемы учитывая специфику синтезаторов.



Процедурные блоки

В целом защелки плохи по нескольким причинам:

1. Нарушение синхронной логики, так как обычно все происходит по фронтам, защелки же срабатывают всегда
2. Их неожиданное появление часто свидетельствует о том, что программист работает с HDL также как с обычным языком программирования
3. При нежелательном синтезе они могут увеличивать задержки сигнала





Чтобы избежать защелки в предыдущем коде, можно например заменить его на:

```
always @(posedge b) begin
    if (a) res = 1;
    else res = 0;
end
```

// При смене b с 0 на 1 блок сработает и изменит значение res, то есть мы по сути защелку заменили триггером.

Это наверное самая верная рекомендация, так как современные синтезаторы и FPGA в разы лучше работают с триггерами

В Verilog существует 3 типа присваивания:

- `assign`
- `<=`
- `=`

Операторы `assign` должны использоваться вне операторов `always`. Вычисляются одновременно с остальной логикой.

Знак равенства `=` означает блокирующее присваивание, а `<=` означает неблокирующее (также известное как одновременное) присваивание.

Настоятельно рекомендуется отчетливо различать, когда использовать `<=` и `=`, иначе код будет работать в симуляции, но на практике нет.

Чтобы было чуть понятнее далее будут приведены небольшие рекомендации.



1. Используйте `always_ff @(posedge clk)` (только в SystemVerilog) или `always @(posedge clk)` и неблокирующие присваивания для моделирования последовательной логики.
2. Используйте непрерывные присваивания для моделирования простой комбинационной логики.
3. Используйте `always_comb` (только SystemVerilog) или `always @(*)` и блокирующие присваивания для моделирования более сложной комбинационной логики, но учтите что в Verilog есть риск появления защелок.
4. Не присваивайте значение одному и тому же сигналу в разных операторах `always` или непрерывных присваиваниях.

```
always @(posedge clk)
begin
```

```
    n1 <= d;
```

```
    q <= n1;
```

```
end
```

```
assign y = s ? d1 : d0;
```

```
always @(*)
```

```
begin
```

```
    p = a ^ b;
```

```
    g = a & b;
```

```
    s = p ^ cin;
```

```
    cout = g | (p & cin);
```

```
end
```



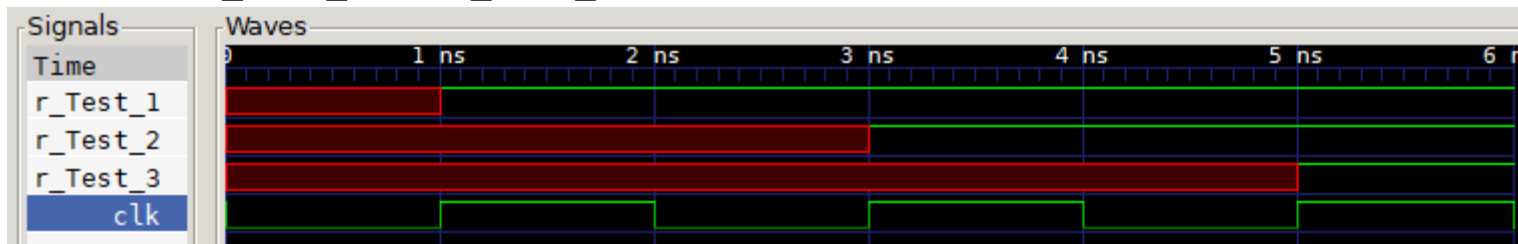
Присваивания

```
r_Test_1 <= 1'b1;  
r_Test_2 <= r_Test_1;  
r_Test_3 <= r_Test_2;
```



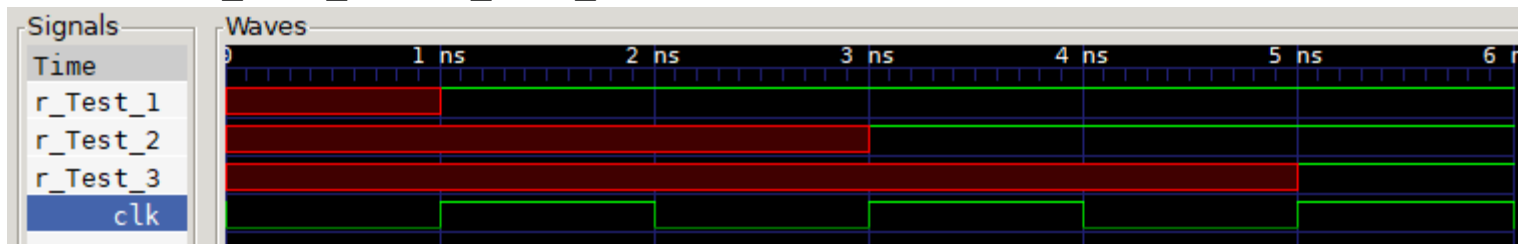
Присваивания

```
r_Test_1 <= 1'b1;  
r_Test_2 <= r_Test_1;  
r_Test_3 <= r_Test_2;
```



Присваивания

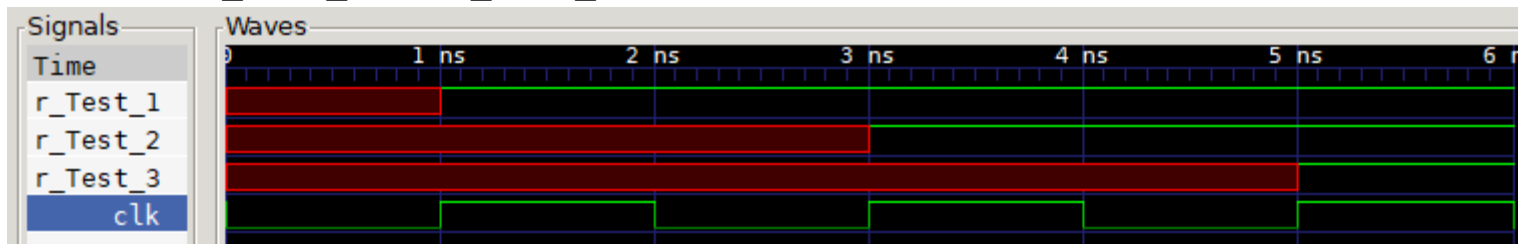
```
r_Test_1 <= 1'b1;  
r_Test_2 <= r_Test_1;  
r_Test_3 <= r_Test_2;
```



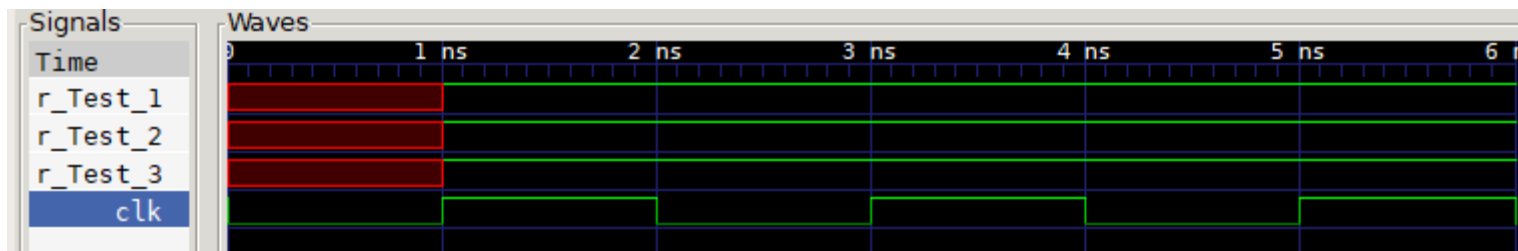
```
r_Test_1 = 1'b1;  
r_Test_2 = r_Test_1;  
r_Test_3 = r_Test_2;
```

Присваивания

```
r_Test_1 <= 1'b1;  
r_Test_2 <= r_Test_1;  
r_Test_3 <= r_Test_2;
```

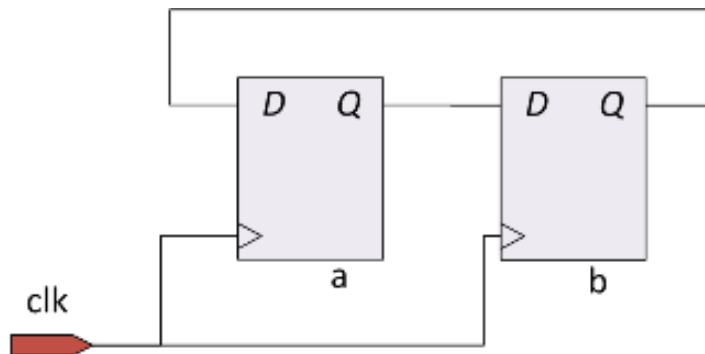
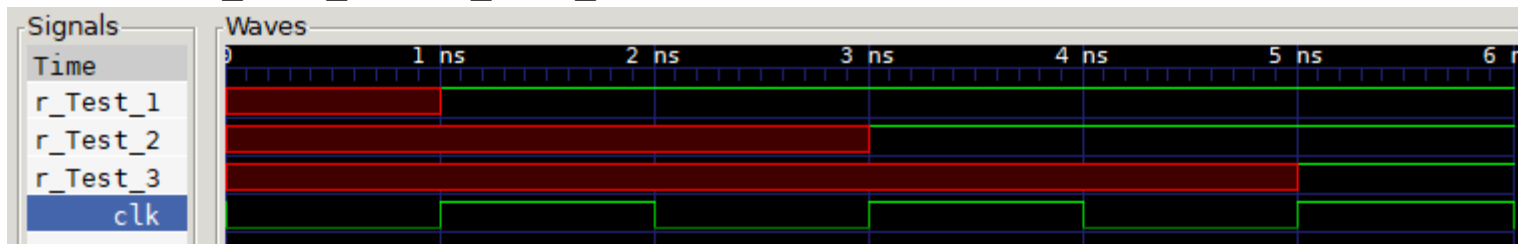


```
r_Test_1 = 1'b1;  
r_Test_2 = r_Test_1;  
r_Test_3 = r_Test_2;
```



Присваивания

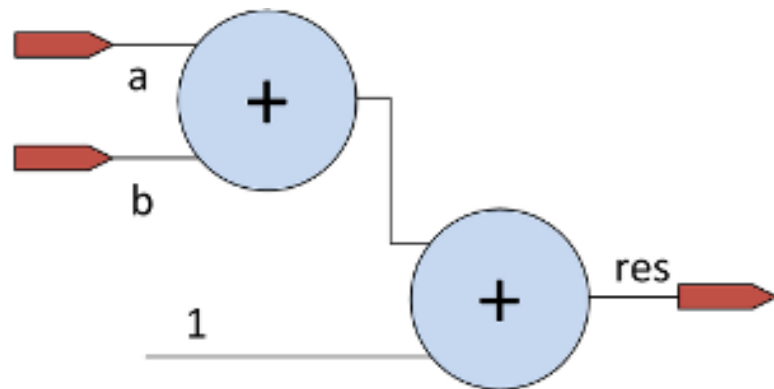
```
r_Test_1 <= 1'b1;  
r_Test_2 <= r_Test_1;  
r_Test_3 <= r_Test_2;
```



Тип данных **wire**

- Не хранит состояние.
- Значение на входе схемы автоматически меняет значение на выходе.
- Используется вместе с операцией непрерывного присваивания **assign**.

```
module comb
(
input wire [7:0] a,b,
output [7:0] res // тип wire
устанавливается по-умолчанию, но можно
написать output wire [7:0] res
);
    assign res = a+b+1;
endmodule
```



Wire

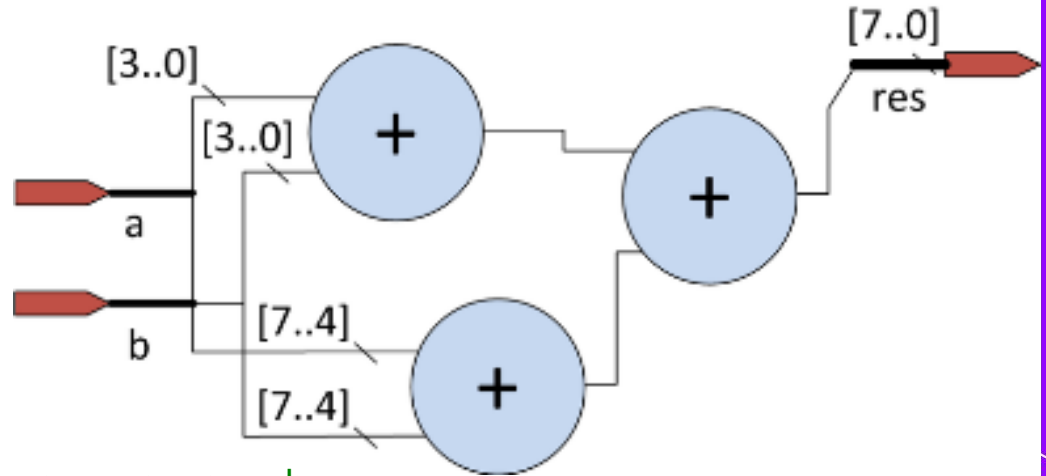
```
module comb
(
  input wire [7:0] a,b,
  output [7:0] res
);
```

```
  wire [3:0] x,y;
```

```
  assign x = a[3:0] + b[3:0];
  // складываются 4 младших разряда на шинах a и b
```

```
  assign y = a[7:4] + b[7:4];
  // складываются 4 старших разряда на шинах a и b
```

```
  assign res = x+y;
endmodule
```



Тип данных **reg**



- Хранит состояние.
- Используется в процедурных блоках.

Вообще ошибочно считать, что reg это регистр. По идее reg будет все, что использует присвоение $\leq$ или $=$.

Таким образом сигнал типа reg может быть выходом триггера, защелки или комбинационной логики, в зависимости от списка чувствительности и оператора внутри always-блока.